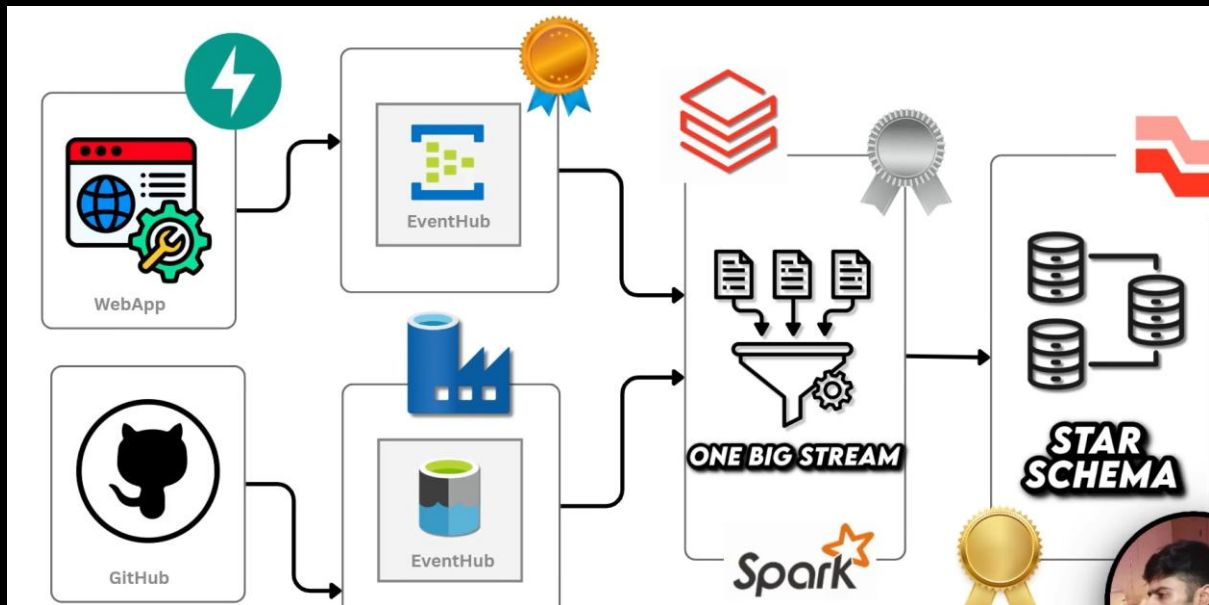
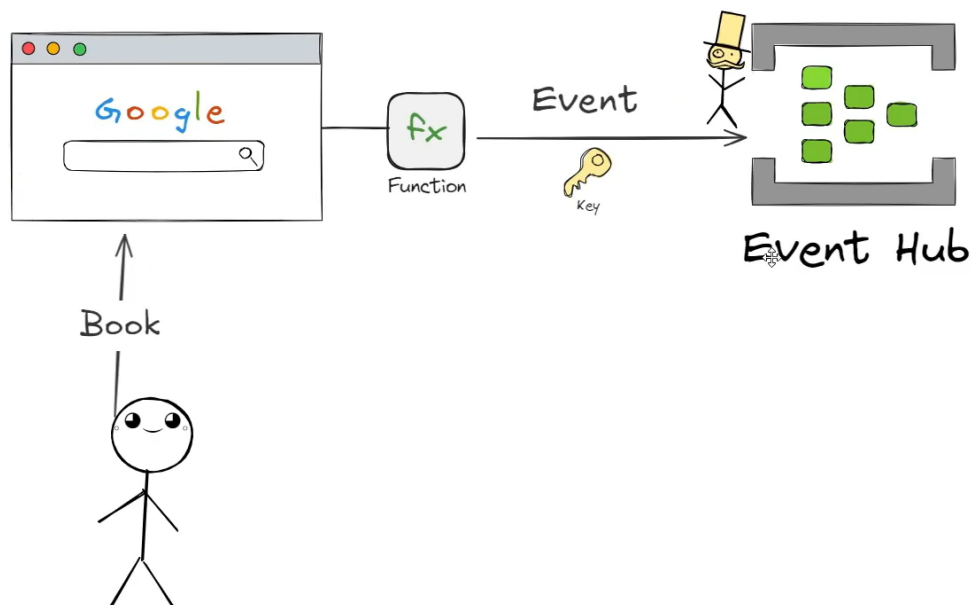


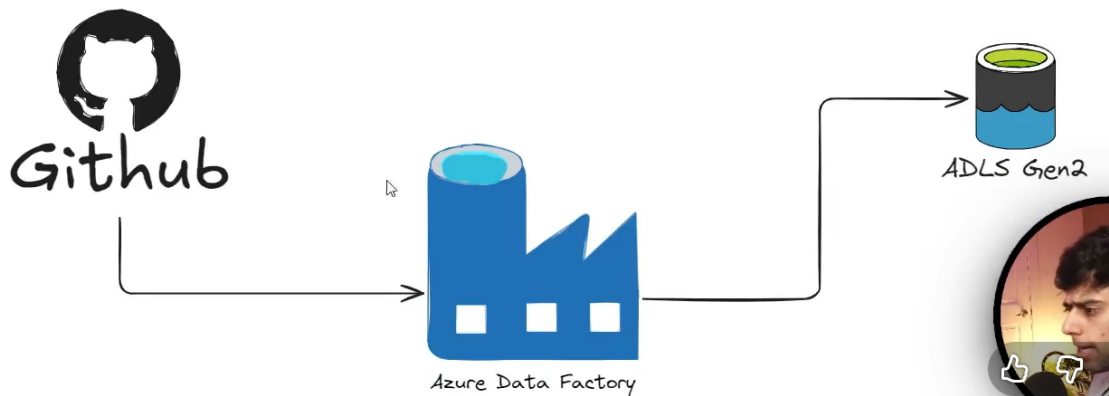
Project 2



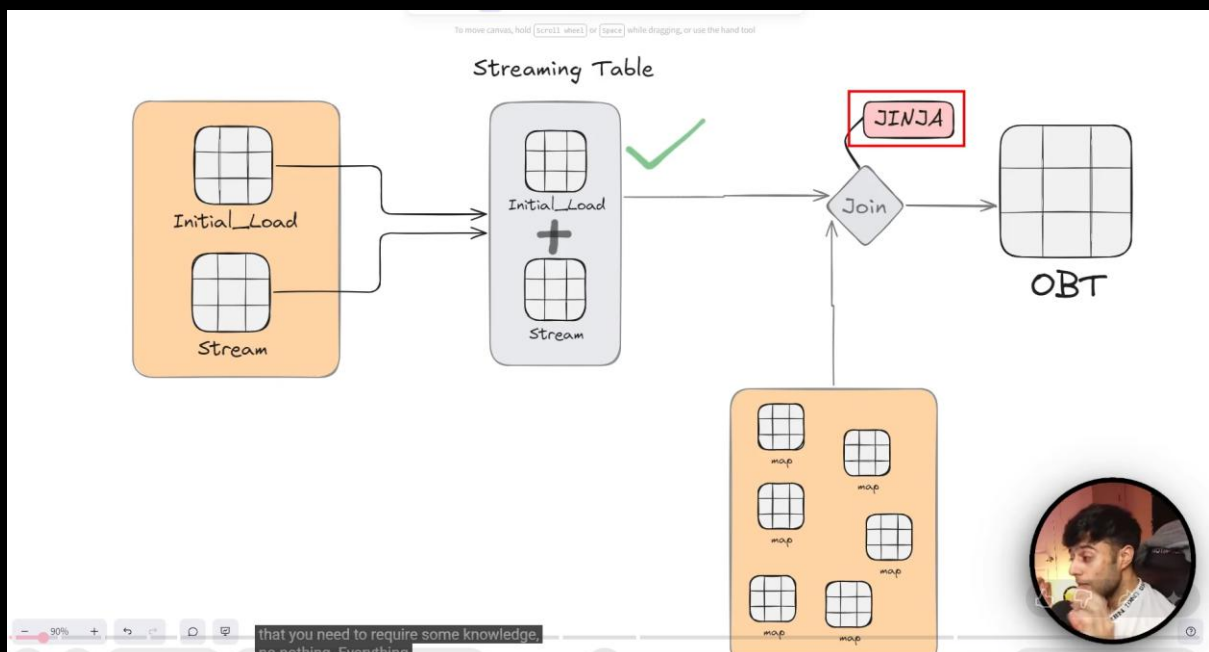
WebApp To EventHub



ADF - DataLake



MetaData: You do not need to tweak or modify your code for the future enhancement if you have new requirement in the future, you just need to make changes in the config file and that will handle everything in the code.



You need to write code one time and reuse it to multiple time

1. Pub-Sub (Publisher-Subscriber)

- **Mechanism:** The **Publisher** sends a message to a "Topic" without knowing who will receive it. **Subscribers** "subscribe" to that topic to receive updates.
- **Relationship:** One-to-Many. One message can be read by multiple different systems (e.g., a "New Order" event goes to the Shipping system AND the Email system).
- **Azure Tool:** **Azure Service Bus** or **Event Grid**.

2. Pro-Con (Producer-Consumer)

- **Mechanism:** The **Producer** creates data and sends it to a buffer or queue. The **Consumer** pulls the data from that queue to process it.
- **Relationship:** Usually One-to-One (per message). Once a consumer processes a message from a standard queue, it's typically gone.
- **Azure Tool:** **Azure Storage Queues**.

3. Azure Event Hubs (The Middle Ground)

- **Definition:** A big data streaming platform and event ingestion service.
- **Why it's special:** It acts like a "tape recorder." It stores the events so multiple consumers can read the data at their own pace without deleting it for others.

Interview Questions on this Topic:

1. "When would you use Azure Event Hubs instead of Azure Service Bus?"
 - *Answer:* Use **Event Hubs** for high-volume telemetry and data streaming (millions of events). Use **Service Bus** for high-value "financial" messaging where you need features like "sessions" or "dead-lettering."
2. "What is a 'Partition' in Event Hubs, and why is it important?"
 - *Answer:* Partitions are used for **parallelism**. More partitions allow more consumers to read data simultaneously, increasing the overall throughput (speed) of your pipeline.
3. "Explain the 'Consumer Group' concept in Event Hubs."
 - *Answer:* A Consumer Group is a "view" of the entire hub. It allows different applications to read the same stream of data independently at their own offset (position).

What is Event Hub?

- **Azure Event Hubs** is a **Platform-as-a-Service (PaaS)** offering from Microsoft.
- It's designed to **collect, stream, and process large volumes of events/data in real time** (like logs, telemetry, IoT signals, or clickstreams).
- Because it's PaaS, **you don't manage servers, clusters, or infrastructure**. Microsoft takes care of scaling, hosting, patching, and availability.
- You just **use the service**: send events in, consume them out, and focus on your application logic.

Event Hub is a platform as a service you do not need to manage the infrastructure you do not need to worry about the hosting but in apache kafka you have to worry about the infrastructure like cluster size, machine size, vm size. In azure it will take care of infrastructure hosting you just need to use the service. That is the biggest advantage of using event hub in our industry.

What is Apache Kafka?

- **Apache Kafka** is an **open-source distributed event streaming platform**.
- It's extremely powerful and widely used for building real-time data pipelines.
- But Kafka is **self-managed**: you (or your team) must handle infrastructure like:
 - Cluster size
 - Machine/VM sizing
 - Storage and replication
 - Monitoring and scaling
- This gives flexibility but also adds **operational overhead**.

◆ Key Differences (Event Hub vs Kafka)

Feature	Apache Kafka (Standard/On-Prem)	Azure Event Hubs (PaaS)
Infrastructure	You must manage the Infra . You worry about Cluster size, VM types, RAM, and storage disks.	Zero Infra management. Azure handles all the underlying hardware and hosting.
Setup Time	Can take days to configure a production-ready, high-availability cluster.	Can be created in seconds via the Azure Portal or CLI.
Scaling	You have to manually add more "Brokers" (servers) to the cluster to handle more load.	It offers Auto-Inflate . As your data traffic grows, Azure automatically scales the "Throughput Units" for you.
Maintenance	You are responsible for patching the OS, updating Kafka versions, and hardware failures.	Fully Managed. Microsoft handles all security patches, version updates, and hardware reliability.

◆ The Point You Highlighted (Industry Advantage)

The **biggest advantage of Event Hubs** is exactly what you said:

"Event Hub is a platform as a service. You do not need to manage the infrastructure or worry about hosting. In Apache Kafka, you have to worry about infra like cluster size, machine size, VM size. In Azure, it takes care of infra hosting — you just need to use the service."

This is why many companies prefer Event Hubs when they want **speed, simplicity, and reliability without building a Kafka ops team**. Kafka is powerful, but Event Hubs removes the **burden of infrastructure management**, which is a huge win in enterprise environments.

- **Use Event Hubs** if you want a **ready-to-go, managed streaming service**.
- **Use Kafka** if you want **full control and customization**, but are ready to handle infra complexity.

Asas

Q If I have a legacy application written for Kafka, do I have to rewrite it to use Azure Event Hubs?"

- **Answer:** No. Event Hubs provides a **Kafka-compatible endpoint**. You only need to change the connection string/configuration to point to Event Hubs, and your Kafka-based app will work seamlessly.

QIn which scenario would you still choose Apache Kafka over Event Hubs?"

- **Answer:** If the project requires **Long-term data retention** (storing streams for months/years) or if the company has a **Multi-cloud strategy** and wants to avoid "Cloud Vendor Lock-in."

QWhat is 'Auto-Inflate' in Event Hubs?"

- **Answer:** It is a feature that automatically increases the number of Throughput Units when your traffic exceeds the set limit, preventing your data pipeline from being throttled or dropped.

QWhy is Event Hubs called 'Distributed'?"

- **Answer:** Because it uses **Partitions**. Data is spread across multiple storage units (partitions), allowing multiple consumers to read different parts of the data at the same time, which provides high speed and scale.

Part 1: EVENT HUB

EVENT HUB AND EVENT HUB NAMESPACE:

Event Hub Namespace (The Container)

- **Definition:** A logical container that holds one or more Event Hubs.
- **Level:** High-level management boundary where you set **Pricing Tier** (Standard/Premium) and **Region**.
- **Throughput Units (TUs):** Defined at the Namespace level. All Event Hubs inside share these resources.
- **Security:** Shared Access Policies (SAS) set here apply to all Hubs inside.

2. Event Hub (The Entity/Topic)

- **Definition:** The actual data stream or "Topic" where messages are sent and stored.
- **Level:** Child entity inside the Namespace.
- **Partitioning:** You define the number of **Partitions** (data lanes) here.
- **Retention:** You set the **Retention Period** (how many days to keep data) at this level.

❓ **If I have 5 Event Hubs in one Namespace and I set the Namespace to 1 Throughput Unit (TU), what happens?"**

- **Answer:** All 5 Event Hubs will **share** that 1 TU (1 MB/s ingress). If one Event Hub uses all the bandwidth, the others will be throttled. You must increase the TUs at the Namespace level to support more traffic across all hubs.

❓ **"Can an Event Hub exist without a Namespace?"**

- **Answer:** No. The Namespace provides the unique FQDN (Fully Qualified Domain Name) endpoint used by producers and consumers to connect.

❓ **"Why would I create multiple Event Hubs in a single Namespace instead of just one?"**

- **Answer:** To organize different data streams that share the same environment or project requirements (e.g., one for logging-data, one for transaction-data) while keeping them logically separate for different consumers.

❓ **"How does the 'PaaS' benefit apply specifically to the Namespace?"**

- **Answer:** In the PaaS model, you don't worry about the server running the Namespace. If you need more power, you just move a slider in the Azure Portal to increase TUs, and Azure handles the underlying hardware scaling instantly.

Home

Create Event Hub

Event Hubs

Basics Capture Review + create

Event Hub Details

Enter required settings for this event hub, including partition count and message retention.

Name * ⓘ ✓

Partition count ⓘ 1

Retention

Configure retention settings for this Event Hub. [Learn more](#)

Cleanup policy ⓘ ▼

Retention time (hrs) * ⓘ ✓
min. 1 hour, max. 168 hours (7days)

Partition count:

Azure Event Hubs: Partitions

Definition: A **Partition** is a "physical lane" or sub-queue within an Event Hub. Think of it like lanes on a highway—the more lanes you have, the more "traffic" (data) can flow through at the same time.

Key Concepts from the Image:

- **Partition Count:** This setting determines how many parallel lanes your data is split into.
- **Parallelism:** Each partition is independent. This allows multiple **Consumers** to read from different partitions simultaneously, significantly increasing processing speed.
- **Scaling:** In the Standard Tier, you can typically choose between **1 to 32 partitions**.
- **Data Ordering:** Data sent to the *same* partition stays in the exact order it arrived. If you send data across *different* partitions, the global order is not guaranteed.

How do partitions help in "Load Balancing"?

- **Answer:** Event Hubs automatically distributes incoming messages across all available partitions (using Round-Robin) unless a Partition Key is specified. This ensures no single partition is overloaded while others are idle.

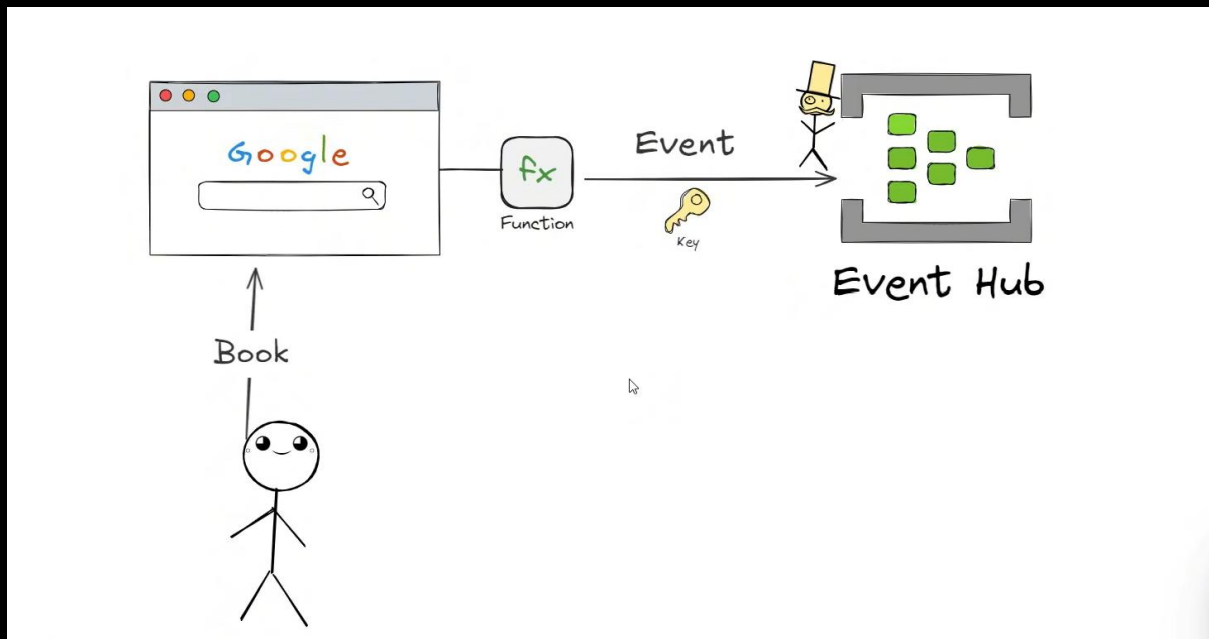
Q4: Does having more partitions increase the cost?

- **Answer:** No. In the Standard Tier, you pay for **Throughput Units (TUs)**, not the number of partitions. However, more partitions allow you to utilize higher TUs more effectively.

Making the two identity by using the shared access policy inside the event hub not on the namespace level

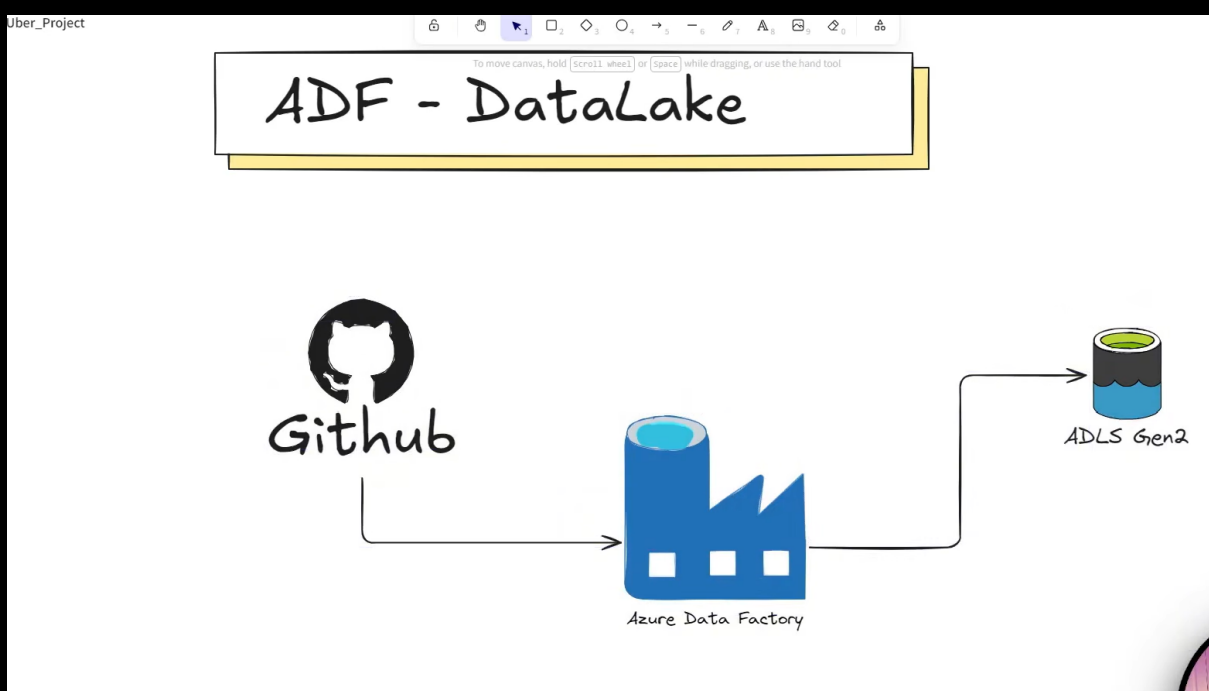
Dasasass

Send events to azure event hub



Its done in VS code

PART 2 :DATA FACTORY



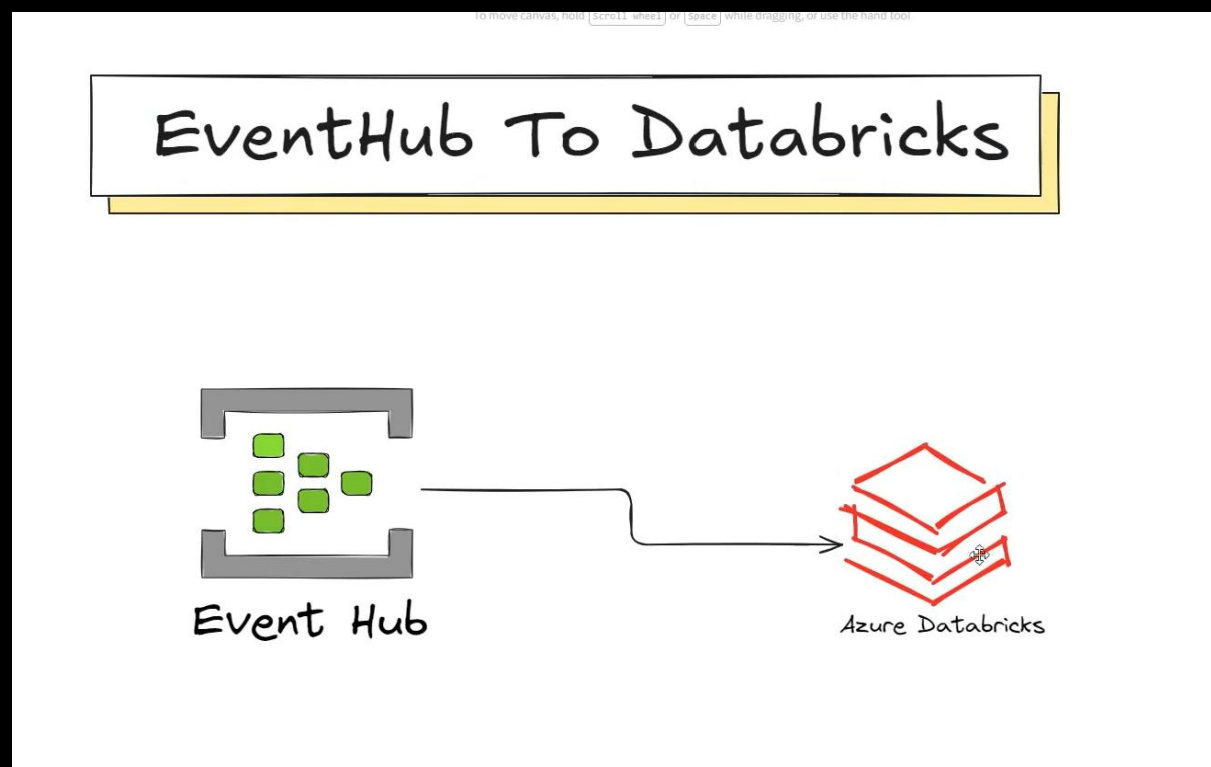
Mene kuch files li github se jisme ek main file thi baaki map files thi jisme additional data tha

Your organization create this file because it is related to the data which is been generated on real time basis and it is important because these are the mappers means mapping files that provide context for your event so that's why we have mapping tables

Mene linked service me http use kiya me use github ya fir rest bhi use kr skti thi lekin rest mujhe json me data deta http is a kind of linked service whenever I want to make any api call

Meta data config ingestion

Part 3: DataBricks



Databricks me dusra account liya uska azure wala nhi liya ushme mene catalog banaya sabse phele

SDP DATABRICKS:

1. What are Spark Declarative Pipelines (Delta Live Tables)?

In traditional Spark, you write **Imperative** code (Step 1: Read, Step 2: Filter, Step 3: Write). In **Declarative** pipelines, you simply define the **Source**, the **Transformation**, and the **Destination**. Spark (via DLT) handles the "How" (cluster management, retries, and dependencies) automatically.

Simple Analogy: > * **Imperative (Standard Spark):** Giving someone a recipe and watching them cook every step.

- **Declarative (DLT):** Ordering a pizza. You define what you want (Toppings, Size), and the kitchen handles the oven and the timing.
-

2. Why did it come into the Market? (The Core Reason)

Before DLT, data engineers spent a lot of their time fixing pipelines and only building them.

- **The Problem:** In standard Spark, you have to manually handle checkpointing, schema evolution, data quality checks, and retries.
 - **The Solution:** Databricks launched DLT to make pipelines **Reliable, Maintainable, and Scalable** by automating the "boring" parts of data engineering.
-

3. Key Features (The "Why Use It")

1. **Automatic Dependency Mapping:** If you have 10 tables, DLT builds a **DAG (Directed Acyclic Graph)**. It knows Table B depends on Table A, so it runs them in the correct order.
 2. **Expectations (Data Quality):** You can set "Expectations" (e.g., `CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW`). If data is bad, DLT handles it based on your rule.
 3. **Medallion Architecture Support:** It is designed to move data seamlessly from **Bronze Silver Gold**.
 4. **Simplified Streaming & Batch:** You can switch between Batch and Streaming by just changing one keyword (`STREAMING LIVE TABLE` vs `LIVE TABLE`).
 5. **Self-Healing:** If a cluster fails or a spot instance is reclaimed, DLT automatically restarts the pipeline from where it left off.
-

4. Difference: Standard Spark vs. Declarative (DLT)

Feature	Standard Spark (Imperative)	Spark Declarative (DLT)
Code Style	df.write.save()	@dlt.table (Python) or CREATE LIVE TABLE (SQL)
Data Quality	Manual checks/Unit tests	Built-in "Expectations"
Recovery	Manual Checkpointing	Automatic Checkpointing
Infrastructure	You manage the cluster	DLT manages the "Enhanced Autoscaling"
Lineage	Hard to track	Visual UI shows data flow automatically
Export to Sheets		

5. When should you use it?

- **Use DLT when:** You are building a complex "Medallion" lakehouse, you need built-in data quality monitoring, and you want to reduce "Operational Overhead."
 - **Don't use DLT when:** You need very low-level control over Spark configurations or if you are running a very simple, one-off script.
-

6. Top Interview Questions (The "Tough" Ones)

Q1: What are "Expectations" in DLT?

- **Ans:** They are data quality constraints. There are three types: RETAIN (log error, keep row), DROP (log error, delete row), and FAIL (stop the whole pipeline if data is bad).

Q2: What is the difference between a "Live Table" and a "Streaming Live Table"?

- **Ans:** A **Live Table** processes the entire dataset (Batch). A **Streaming Live Table** only processes the "new" data since the last refresh (Incremental/Streaming).

Q3: How does DLT handle Schema Evolution?

- **Ans:** It has built-in support for schema evolution. If a new column is added to the source, DLT can automatically detect and update the schema without the pipeline failing.

Q4: Can we mix SQL and Python in the same DLT pipeline?

- **Ans:** Yes! You can have some tables defined in SQL files and others in Python files within the same DLT pipeline.

. The "Quality Check" Problem (Data Validation)

Manual Spark:

If you send a String in a Weight column that should be an Integer, standard Spark will either:

1. Make the whole column NULL.
2. Throw an error and kill the whole 2-hour long job.
3. You have to write 50 lines of if-else or filter logic manually to check every column.

DLT (The Declarative Way):

You use **Expectations**. It tracks it like a "Data Police."

- **How it knows:** You define a constraint like `@dlt.expect_or_drop("valid_weight", "weight > 0")`.
- **What happens:** DLT automatically monitors the data. It gives you a **Dashboard** showing exactly how many rows failed. You don't have to write "tracking" logic; DLT has a built-in event log that records every bad record.

2. The "Checkpointing" Headache

Manual Spark:

You have to manually define a path: `.option("checkpointLocation", "/mnt/telemetry/checkpoints")`.

- If you change your code logic slightly, the checkpoint might become incompatible.
- If you delete the checkpoint folder by mistake, your pipeline might re-process 2 years of old data.

DLT (The Declarative Way):

DLT manages the **State**. You don't give a path. DLT knows exactly which file was read last.

- **Internal Management:** It stores the "offset" (position) in its own managed metadata.

- **Advantage:** If the pipeline fails, DLT looks at its internal "State Store" and resumes from the exact micro-second it stopped. You don't manage folders; you just click "Start."
-

3. Why "Live Tables"? (The Dependency Problem)

Imagine you have 3 tables: **Bronze** $\rightarrow$ **Silver** $\rightarrow$ **Gold**.

- **Standard Spark:** You have to create 3 different notebooks. You have to trigger Notebook 1, wait for it to finish, then trigger Notebook 2. If Notebook 2 fails, you have to manually check if Notebook 1 needs to be re-run.
 - **DLT (Live Tables):** You define them all in one place. DLT creates a **DAG (Visual Map)**.
 - It "lives" because it's aware of its parents.
 - If you update the logic in Silver, DLT knows it only needs to refresh Silver and Gold, not Bronze. It handles the **Orchestration** automatically.
-

4. Schema Evolution: How it works behind the scenes

Manual:

If a new column Country arrives in your JSON, Spark will ignore it unless you stop the stream and update the schema manually.

DLT:

It has **Schema Inference** turned on by default.

- It detects the new column.
 - It automatically updates the target Delta Table's schema (Schema Evolution).
 - It restarts the stream internally to include the new column. **Zero manual intervention.**
-

Interview Questions: Why DLT over Manual Spark?

Q1: "In manual Spark, I can use `.filter()` for quality. Why use DLT Expectations?"

- **Answer:** Manual filters just remove data. **DLT Expectations** provide **Observability**. DLT records the metrics of "Bad Data" in a system table that you can query to see data health over time without writing extra logging code.

Q2: "What is 'Enhanced Autoscaling' in DLT?"

- **Answer:** In standard clusters, scaling is slow. DLT "Enhanced Autoscaling" is optimized for streaming. It adds/removes workers much faster based on the "backlog" (how many files are waiting to be processed), saving cost and time.

Q3: "How does DLT handle the 'Small File Problem'?"

- **Answer:** DLT automatically performs **Auto-Optimize** and **Compact** (Optimize) operations on the Delta tables. You don't have to run manual OPTIMIZE commands to keep the performance fast.

Case 1: Azure Databricks (Professional/Enterprise)

In a real Azure environment, you have several secure, production-grade ways to connect.

1. Access Key (Simplest - Not for Production)

- **Mechanism:** You use the Storage Account's secret "Access Key" directly in your Spark configuration.
- **Best for:** Quick testing or personal projects.
- **How:** You set the Spark configuration `fs.azure.account.key.<storage-account>.dfs.core.windows.net` to your key.

2. Service Principal (Standard Industry Method)

- **Mechanism:** You create an **App Registration** in Microsoft Entra ID (Active Directory), give it "Storage Blob Data Contributor" rights, and use its Client ID and Secret to authenticate.
- **Best for:** Automated pipelines and production workloads.
- **Note:** Always store these secrets in **Azure Key Vault** and link it to a Databricks **Secret Scope**.

3. Managed Identity (Unity Catalog - Modern Way)

- **Mechanism:** You use an **Access Connector for Azure Databricks**. You assign a Managed Identity to the connector, and Databricks uses this identity to talk to storage without any "secrets" or "keys" at all.
- **Best for:** High security and governance using **Unity Catalog**.

Case 2: Databricks Community Edition (Free Public Account)

The Community Edition is **NOT** hosted on your Azure subscription; it is hosted on Databricks' own AWS infrastructure. Because it is outside your Azure tenant, certain features (like Managed Identity) will not work.

1. Access Key (Direct Connection)

- **How:** This is the most reliable way for the free version. You simply grab the Access Key from your Azure Portal and use this code snippet in your notebook:

Python

```
spark.conf.set(
    "fs.azure.account.key.<storage-account-name>.dfs.core.windows.net",
    "<your-access-key>"
)

df = spark.read.csv("abfss://<container-name>@<storage-account-name>.dfs.core.windows.net/file.csv")
```

2. SAS Token (Shared Access Signature)

- **Mechanism:** You generate a temporary SAS token in Azure with an expiration date and specific permissions (Read/Write).
- **Why use it:** It is safer than an Access Key because you can limit how long the access lasts.

3. Service Principal (Cross-Tenant)

- **How:** You can still use a Service Principal, but you must ensure your Azure Storage Firewall allows "Public access from all networks" because the Community Edition's IP addresses are dynamic and external to Azure.

Q1: Why is "Mounting" a storage account (DBFS Mount) now considered a legacy practice?

- **Answer:** Mounting is less secure because anyone with access to the workspace can see the mounted data. The modern standard is to use **Direct Access** (abfss path) with Service Principals or **Unity Catalog External Locations**.

Q2: What is the "abfss" protocol?

- **Answer:** It stands for **Azure Blob File System Storage (Secure)**. It is the driver used by Spark to communicate efficiently with ADLS Gen2 using SSL/TLS.

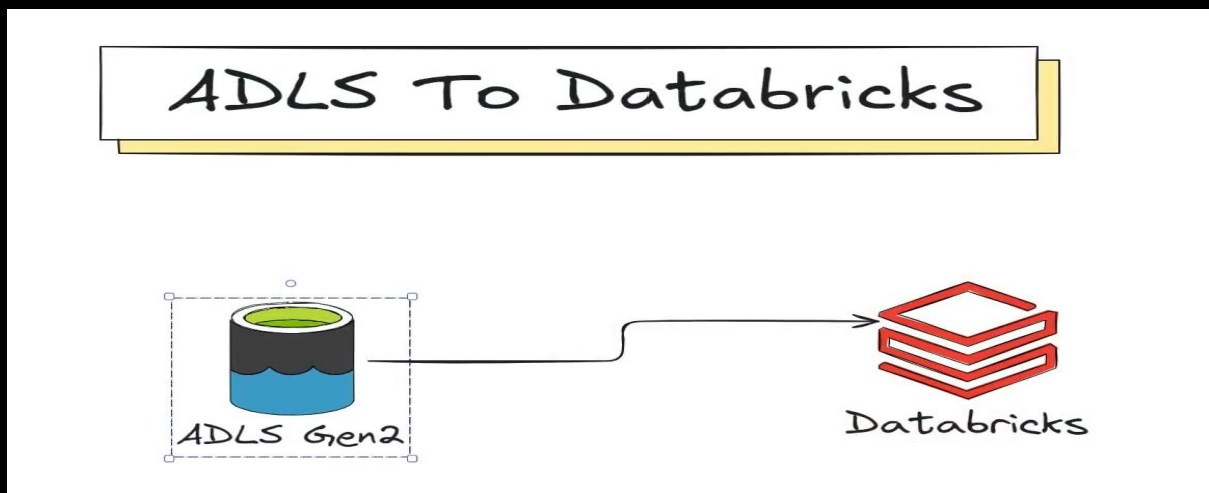
Q3: How do you secure credentials in a Databricks Notebook?

- **Answer:** You should never hardcode keys. Use **Databricks Secret Scopes** to fetch credentials stored in **Azure Key Vault** using `dbutils.secrets.get()`.

Q4: If you are using Databricks Community Edition, can you use Azure VNet injection?

- **Answer:** No. Community Edition is a shared, multi-tenant environment managed by Databricks on AWS. VNet injection is only available in the paid Azure Databricks tiers.

DATABRICKS:



mene apne storage containers ko databricks se connect krne ke liye shared access token banaye container level pe

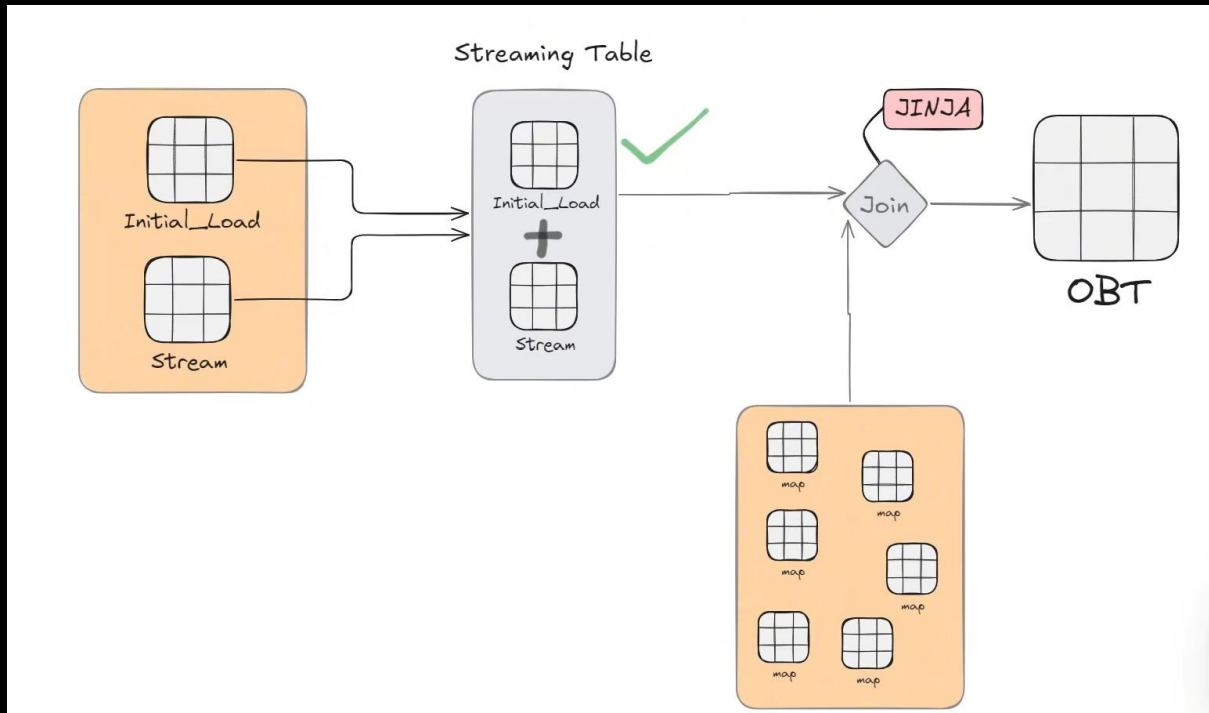
He explains that PySpark is intended for processing large datasets (like fact tables), whereas mapping files are static files where pandas is more appropriate when working directly with a SAS token from the data lake.

whenever you want to read the mapping files, mapping files we do not need to directly create a *pispark* data frame. Why? Because whenever you want to create a *pispark* data frame, you create *pispark* data frame because there's a reason behind it. When you are working with a big data which is a scenario with the data that we are receiving in the real time, so that's why we created the *pispark* data frame because fact tables are big data like big tables. But when we just want to load the mapping files, static files, we should always go with *pandas*'s data frame. Simple, very good. Second thing, we are reading mapping files from data lake and if you want it to directly use it with the SAS token, we have to work with *pandas* data frame.

Mene kya kiya baad me ki apna jo application thin a uske ek or baar run kiya wha se data aya mera uske baad mene jo wo pipeline banayi thi event hub wali databricks me usko dubara se run kiya or jo databricks notbook bnayi thi jisme me file save kra rhi thi usko dubara run kiya ki ye data bhi add ho jaaye dubara

SILVER LAYER

Ek problem aayi like jab me initial load ko stream table ko union ke sath add krungi to wo bhut jada resources or load lega kyuki image agr ek interation hia to theek hai lekin lekin maaan me apni streaming table har 5 min me run krr rhi hu to bhai soch kitna computation resource use hoga or initial load bhut badi table hoti hia phele se hi



Asdasda

Why can't I just use a simple `df.append()` or union? Why do I need this 'Streaming Table' drama?"

Here is the "Real World" reason for your notes:

1. The Problem: The "Re-computation" Trap

Imagine you have **100 GB** of historical data (Initial Load) and **10 MB** of new data coming every minute (Stream).

- **If you use a simple Union/Append in a regular script:** Every time your pipeline runs, Spark looks at the *entire* 100.01 GB. Even if you "append," the transformation logic downstream (like a Join or Group By) has to scan the whole table again to make sure everything is correct. This is like re-reading a whole book just to add one new sentence at the end. It's a waste of money and time.

- *The Checkpoint Headache: If your manual append fails halfway, how do you know where you stopped? You might end up appending the same data twice (Duplication), or missing data (Data Loss).*
-

2. How Streaming Tables (SDP) Solve This

A Streaming Table is not just a table; it is a "Stateful Bookmark."

The Step-by-Step Logic:

1. *The "Unified" Container: You create one Streaming Table.*
2. *Initial Load (The Base): You push the 100 GB into it once. The "Streaming Engine" marks this as "Processed."*
3. *The Append Flow (The Magic): When the 10 MB of new data arrives, the engine looks at its Checkpoint. It says, "I've already handled the 100 GB. I will only touch this new 10 MB."*
4. *Automatic State Management: It merges the 10 MB into the 100 GB incrementally. It doesn't re-scan the old data.*

Uske baad mene kya kiya ki whi dlt banayi or incremental_load or streaming table ko add krke ek pipeline bna di lekin ushme dikkt kya aayi ki jo mera streaming pipeline table thi uske andr ek col ke andr jo data tha wo json format me tha lekin json me nhi tha to kya kiya usko phele json me convert kiya uske baad uske andr jitne bhi col the unko bhi jo incremental_load table ke col the wese banayi uske baad ye streaming table me bhi change krke dono ko append krr diyaa mtlb simple alg se schema define krr diya uske liye bhi ki bhai ye ye data types le or bna de shanti se

Jinja template:

One Big Table(OBT)

Mene whi ek notebook me same jinja template likhi jisse me apni table ko dynamically query krr sku

12:09 PM (<1s)

1

```
import jinja2
from jinja2 import Template
```

2 minutes ago (<1s)

2

```
jinja_config=[
    {
        "table":"uber_project.bronze_layer.stg_rides",
        "select":"uber_project.bronze_layer.stg_rides.*",
        "where":""
    },
    {
        "table" : "uber_project.bronze_layer.map_vehicle_makes map_vehicle_makes",
        "select" : "map_vehicle_makes.vehicle_make",
        "where" : "",
        "on" : "stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id"
    },
    {
        "table" : "uber_project.bronze_layer.map_vehicle_types map_vehicle_types",
        "select" : "map_vehicle_types.vehicle_type,map_vehicle_types.description,map_vehicle_types.base_rate,map_vehicle_types.per_mile,map_vehicle_types.per_minute",
        "where" : "",
        "on" : "stg_rides.vehicle_type_id = map_vehicle_types.vehicle_type_id"
    }
]
```

2 minutes ago (<1s)

3

```
jinja_str="""
    select
    {% for config in jinja_config %}
        {{config.select}}
        {% if not loop.last %}
            ,
        {% endif %}
    {% endfor %}

    from

    {% for config in jinja_config %}
        {% if loop.first %}
            {{config.table}}
        {% else %}
            left join {{config.table}} on {{config.on}}
        {% endif %}
    {% endfor %}

    {% for config in jinja_config %}

        {% if loop.first %}
        {% if config.where != "" %}
            where
        {% endif %}
        {% endif %}

        {{config.where}}
        {% if not loop.last %}
            {% if config.where != "" %}
                and
            {% endif %}
        {% endif %}

    {% endfor %}

"""

template=Template(jinja_str)
rendered_template=template.render(jinja_config=jinja_config)
print(rendered_template)
```

Slowly Changing dimension

Ishme mene apne jo map_cities table thi ushme updated_at col add kiya or adf pipeline dubara chalayi jisse wo map_cties table update ho gyi uske baad databricks me bhi ush table ko dubara se upload krr liya uske baad wo silver pipeline databricks me dubara chalayi stg_rides wali

Next step

Ab iske baad hame jitni bhi maaping files hia usko merging file se join krna haii uske liye use krr rhe hia ham streaming join mtlb watermark technique

Watermark is basically used for statefull tranfomation this is the concept for spark structure streaming

When working with **Spark Declarative Pipelines (SDP)** using **SQL**, you cannot simply use readStream as you would in PySpark.

Instead, to inform Spark that you want to read a table as a **streaming table** within a SQL query, you must add the keyword **STREAM** directly before the table name. Once this keyword is added, the rest of the SQL query syntax remains the same as standard batch processing. The speaker emphasizes that this small addition is all that is required for Spark to recognize the streaming behavior in SQL.

1. What is Watermarking?

In a streaming world, data doesn't always arrive in order. For example, a user's phone might lose internet, and their "Purchase" event happens at **10:00 AM** (Event Time) but only reaches your server at **10:15 AM** (Processing Time).

Watermarking is a threshold that tracks the maximum **Event Time** seen so far and subtracts a "lateness" delay.

```
from

stream (uber_project.bronze_layer.stg_rides)
watermark booking_timestamp delay of interval 3 minutes stg_rides|
```

- **Formula:** $\$Watermark = (Max \setminus Event \setminus Time \setminus Seen) - (Allowed \setminus Lateness \setminus Delay) \$$

2. Why is it necessary? (The Memory Problem)

If you are doing an aggregation (like counting sales every 10 minutes), Spark needs to keep the "state" (the intermediate counts) in its memory.

- **Without Watermarking:** Spark doesn't know if more data for "10:00 AM" is coming. It will keep that window in memory **forever**, eventually leading to an OutOfMemory (OOM) error.
- **With Watermarking:** Once the watermark passes a certain time window, Spark knows: *"I have waited long enough. No more data for this window will be accepted."* It then **deletes that state from memory**, keeping your cluster healthy.

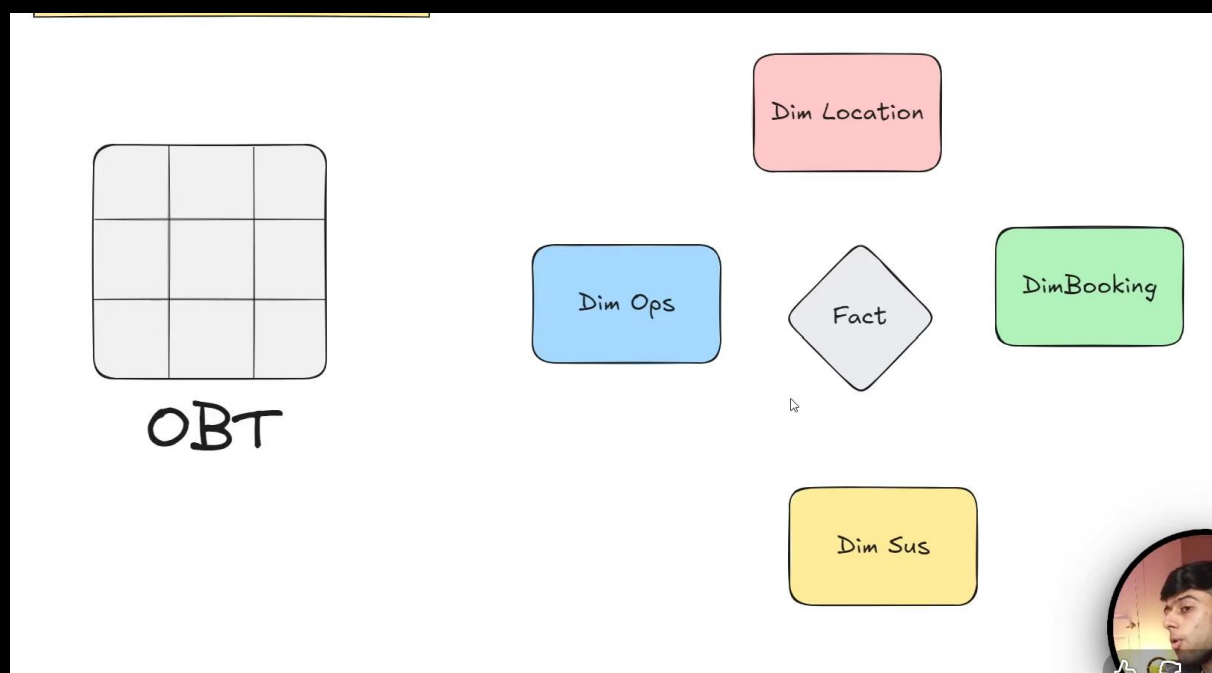
3. How to use it in Spark SQL / PySpark

You use the `.withWatermark()` function. You must specify the **timestamp column** and the **delay threshold**.

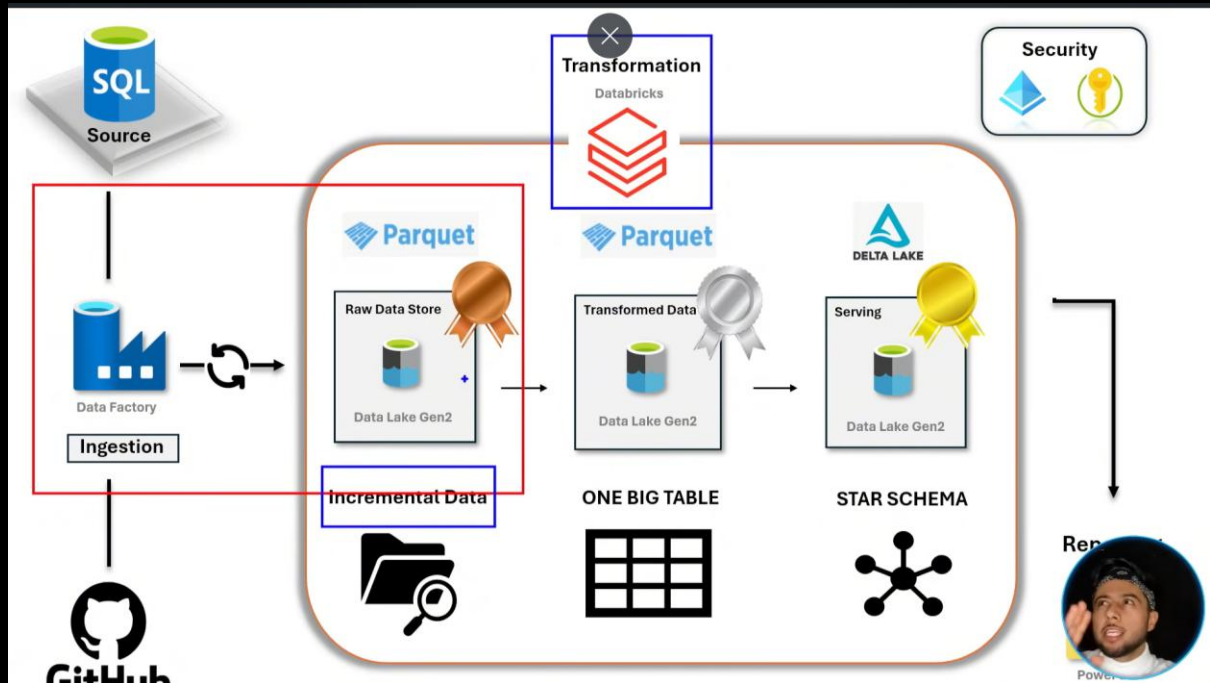
Gold Layer

Ab uske baad jo mere main obt file hia ushme me apni dimestion and fact table bna rhi hu mtlb simply khu to data modelling krr rhi hu by using Auto cdc whi yaar slowly changing dimension me jo banayi thi whi

Ushme mene ek view banay auto cdc col me mtlb ki ush dim passenger table ko hold krne k liye koi source to chihaye rhega naa



Isadadad



Capability	 Microsoft	 aws	 Google
Text Generation	ChatGPT	Bedrock	Bard
Code Generation	Github Copilot	Code Whisperer	CodeY
Image Generation	DALL-E		Imagen
Model Development and Deployment	Azure ML	SageMaker	Vertex AI
Base LLMs	GPT 4	Olympus	Gemini